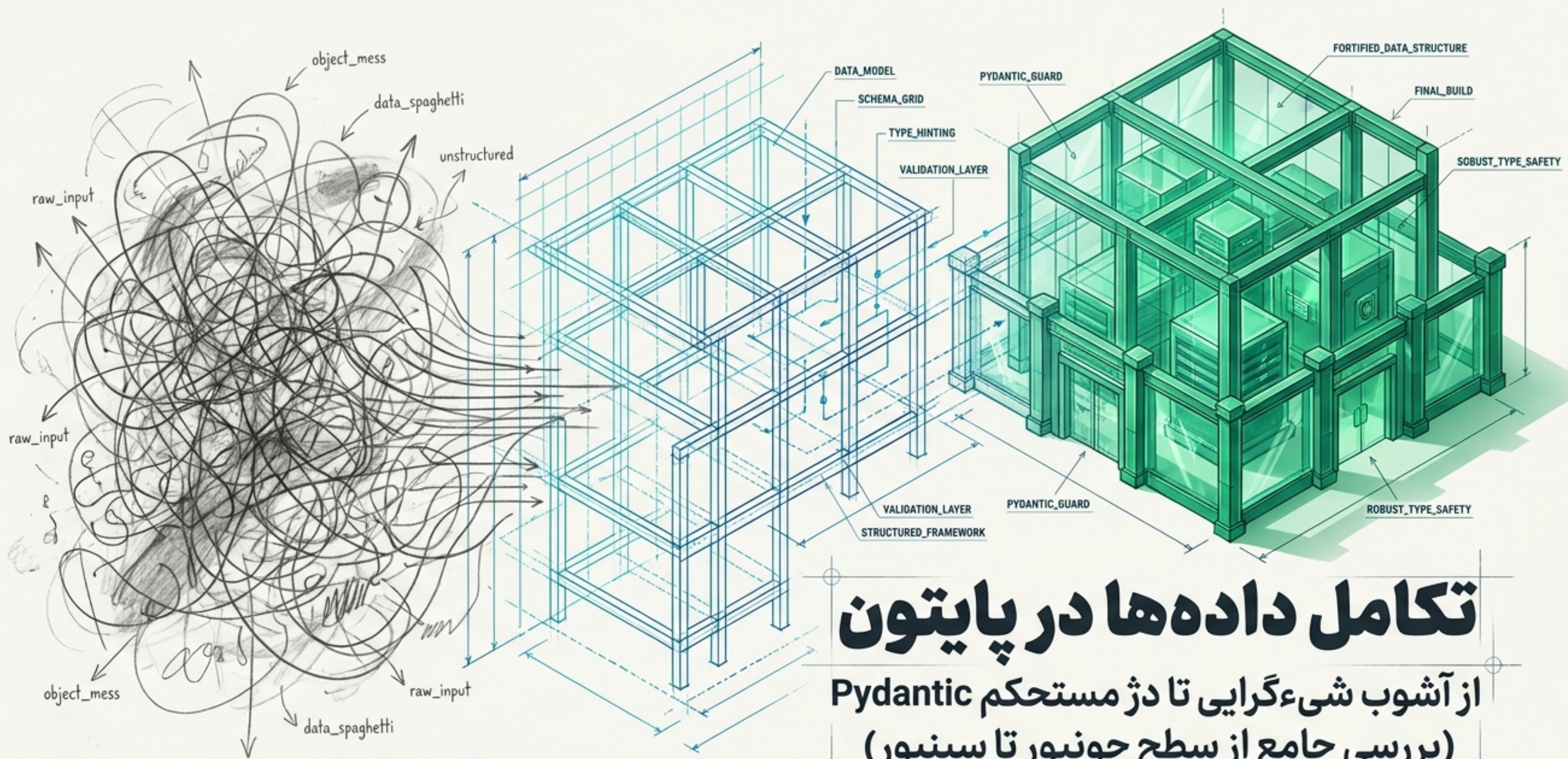


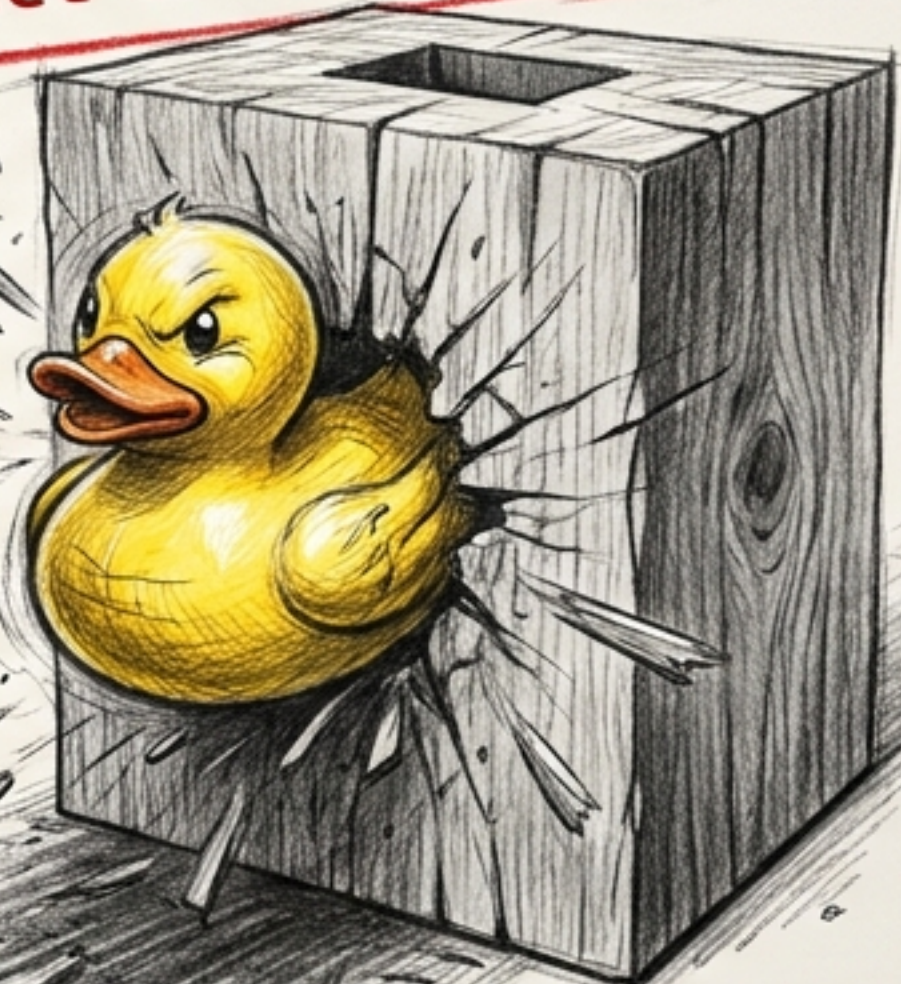


تکامل داده‌ها در پایتون

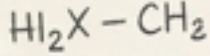
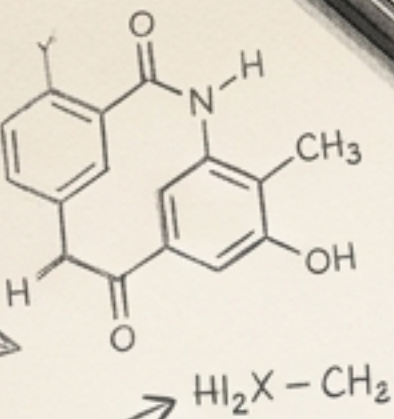
از آشوب شیء‌گرایی تا دژ مستحکم Pydantic
(بررسی جامع از سطح جونیور تا سینیور)

نقطه صفر: تایپینگ اردکی و کدهای تکراری بی پایان

TypeError: 'NoneType' object is not iterable



انعطاف‌پذیری پایتون قدرتمند است، اما در پروژه‌های بزرگ باعث تولید کدهای تکراری و باگ‌های غیرمنتظره در زمان اجرا می‌شود.

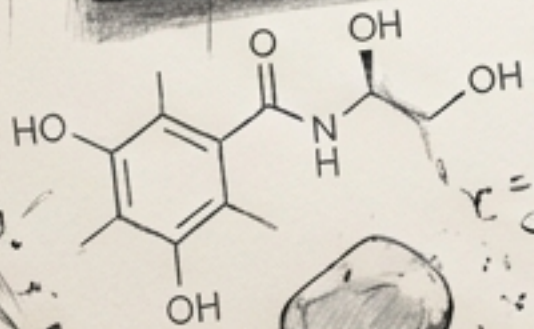


$$(x + R_3, R_4) = (R_1, R_2) + R_3, R_4$$

```
class Molecule:
    def __init__(self, name, charge,
                  symbols, coordinates):
        self.name = name
        self.charge = charge
        self.symbols = symbols
        self.coordinates = coordinates
        self.num_atom = len(symbols)
```

TODO: add methods
validate coords

think about
inheritance



سطح اول: نقشه معماری (Type Hints)

```
class Molecule:
    def __init__(self, name: str, charge: float,
                  symbols: list[str]) -> None:

        self.name = name
        self.charge = charge
        self.symbols = symbols
        self.coordinates = coordinates
        self.num_atom = len(symbols)
```

TYPE ANNOTATION

RETURN
TYPE

TYPE HINT

⚠ Incompatible type: expected :
'Duck', got
'RubberDuck'

راهنما، نه مانع: تایپ هینت‌ها کدهای شما را برای IDE خوانا می‌کنند، اما مانع مفسر پایتون نمی‌شوند.

بررسی ایستا: خطاهای تطبیق نوع داده قبل از اجرای کد کشف می‌شوند.

Type hints analyze without executing.

الگوهای مدرن تایپینگ

✓ انجام دهید (Do) ✓

str | None
سینتکس جدید و خلاصه‌تر

list[int]
استفاده از generic‌های داخلی

object
برای دریافت هر شیء، با حفظ ایمنی

`_IntList: TypeAlias = list[int]`

✗ پرهیز کنید (Don't) ✗

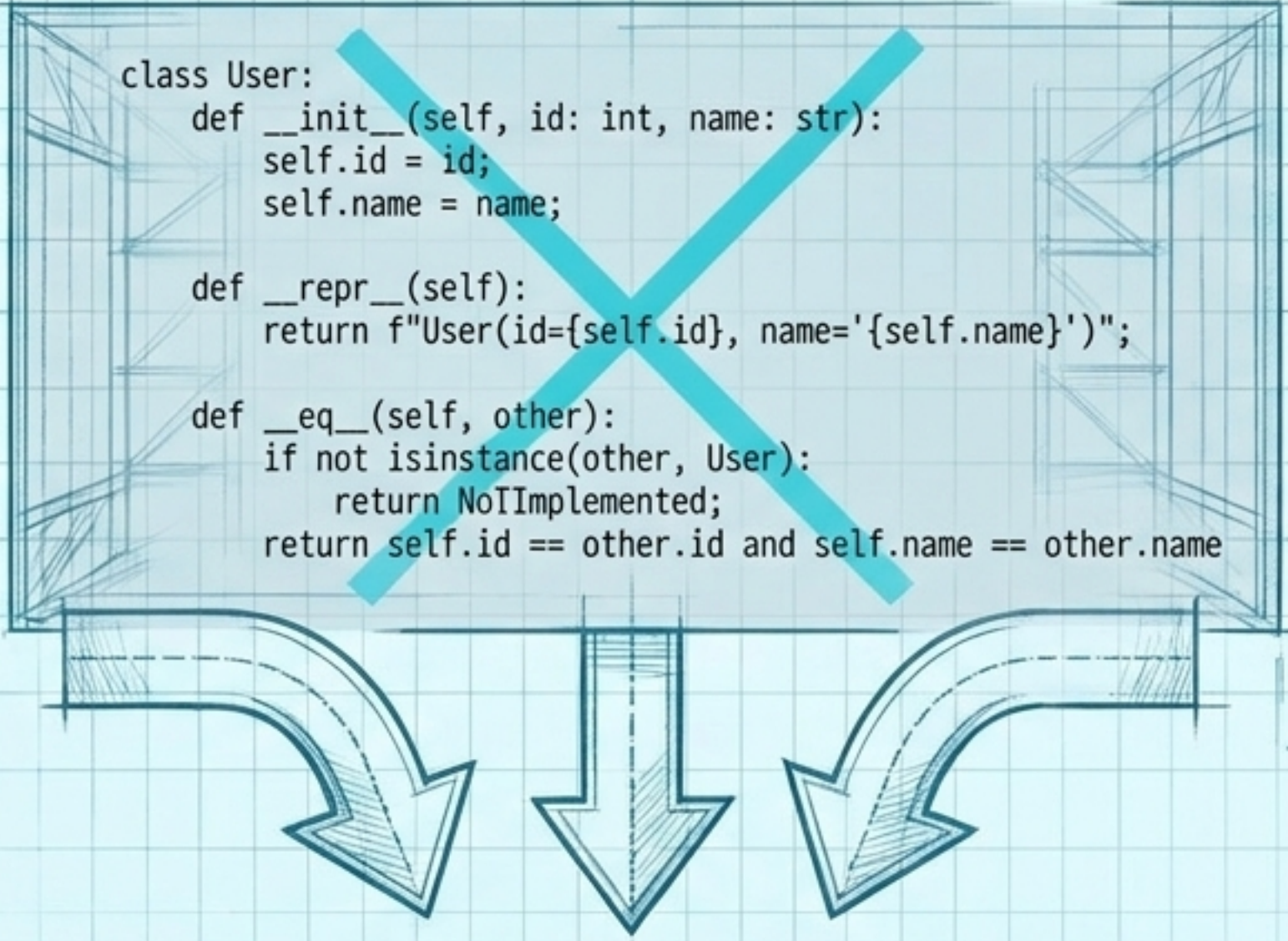
Optional[str] or Union[str, None]

List[int]
منسوخ شده

Any
از بین بردن کامل بررسی تایپ

`_IntList = list[int]`

سطح دوم: ساختار بهینه با Dataclasses



```
class User:
    def __init__(self, id: int, name: str):
        self.id = id;
        self.name = name;

    def __repr__(self):
        return f"User(id={self.id}, name='{self.name}')";

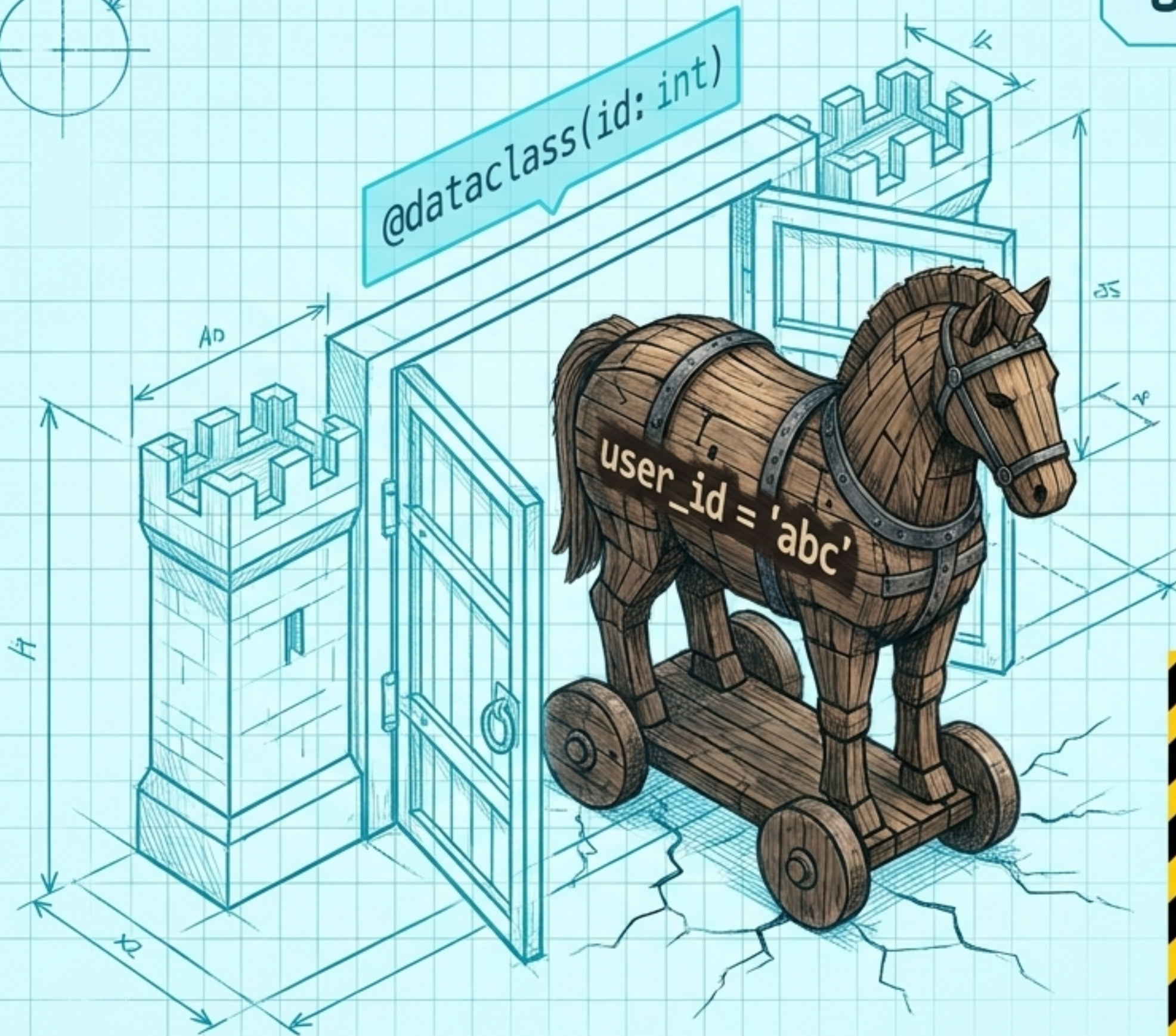
    def __eq__(self, other):
        if not isinstance(other, User):
            return NotImplemented;
        return self.id == other.id and self.name == other.name
```

```
@dataclass
class User:
    id: int
    name: str
```

حذف Boilerplate: پایتون متدهای خسته‌کننده مانند سازنده و چاپ را به صورت خودکار تولید می‌کند.

عملکرد بالا: ایده‌آل برای مدل‌سازی وضعیت‌های داخلی سیستم با سرعت بالا و وزن کم.

اسب تروآ: توهم امنیت در تایپینگ ایستا



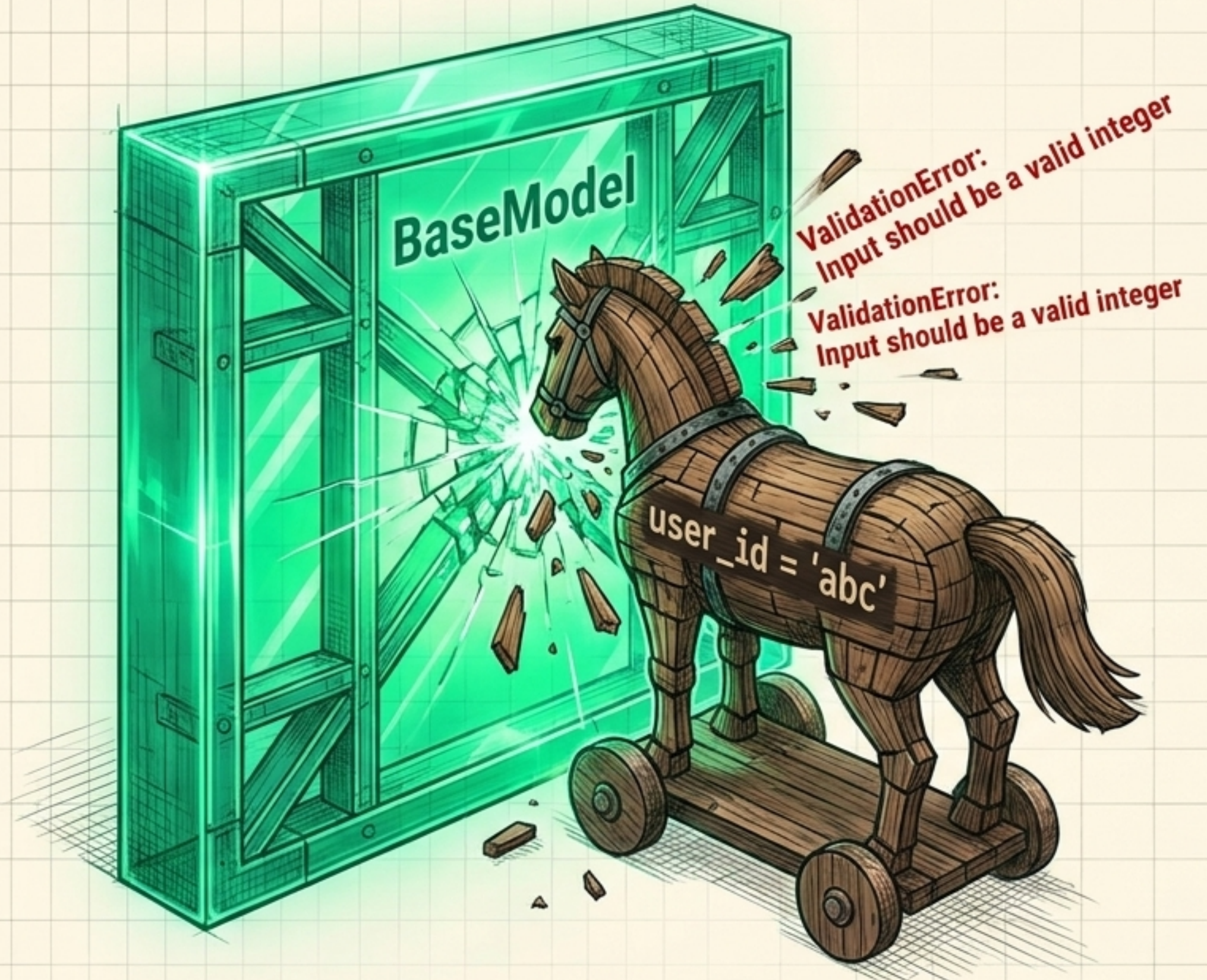
تایپ هینت ها و Dataclass ها داده ها را در زمان اجرا اعتبارسنجی نمی کنند!

اگر داده ها از دنیای بیرون وارد شوند، یک Dataclass به راحتی داده های اشتباه را می پذیرد و باعث فروپاشی در لایه های عمیق تر سیستم می شود.

سطح سوم: دژ مستحکم Pydantic

امنیت در زمان اجرا: Pydantic
تایپ‌هینت‌ها را به قوانین سخت‌گیرانه
در زمان اجرا تبدیل می‌کند.

تبدیل خودکار: مقادیر قابل تبدیل (مثلاً
'5' به 5) را به صورت هوشمند مدیریت کرده و
داده‌های نامعتبر را مسدود می‌کند.



قدرت‌های ویژه Pydantic

تبدیل نوع هوشمند (Type Coercion)
تبدیل ایمن داده‌ها به نوع صحیح.

```
age='5' → age=5
```

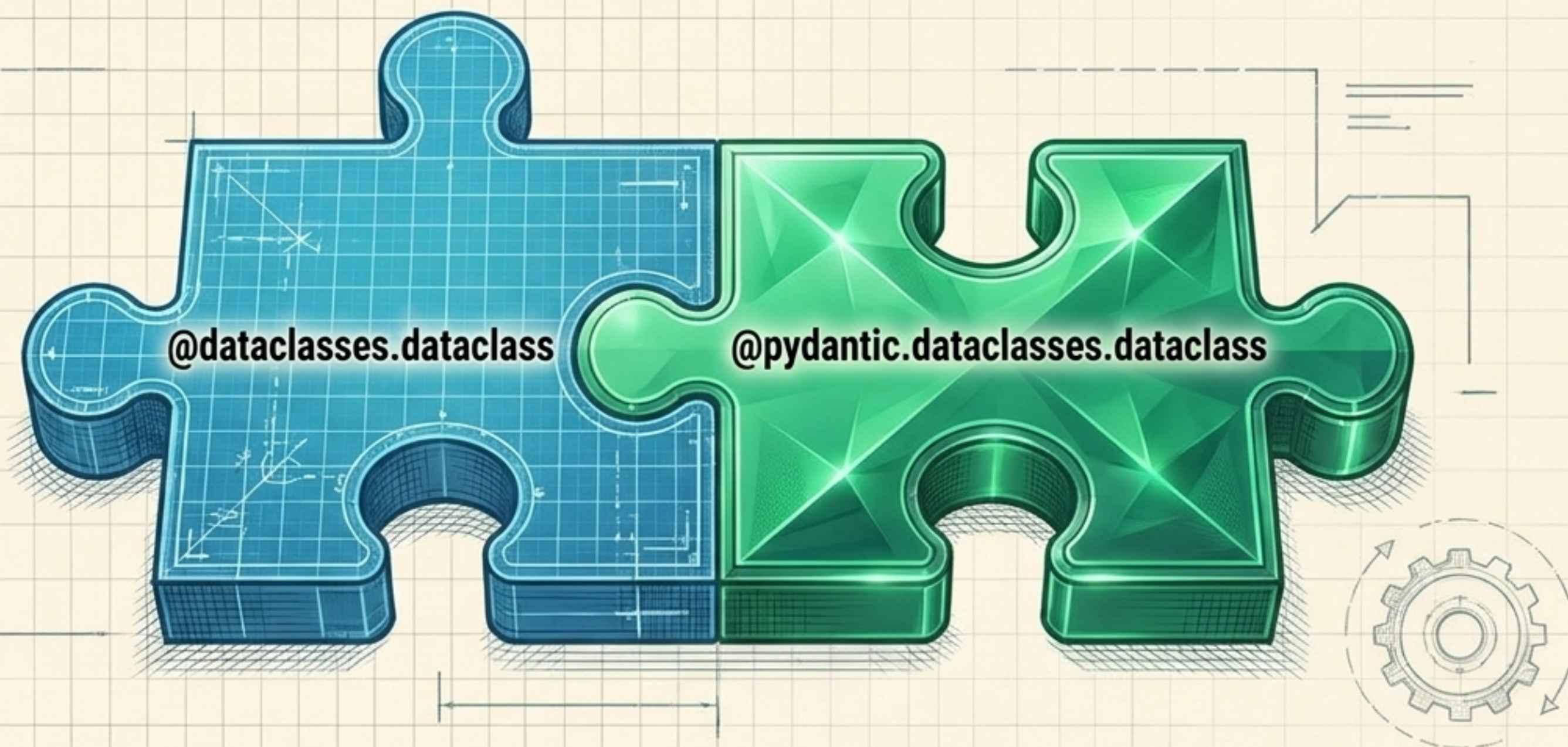
اعتبارسنجی توابع (Validate Call)
استفاده از دکوراتور `@validate_call` روی
آرگومان‌های توابع، بدون نیاز به تعریف مدل.

Pydantic

گزارش خطای دقیق (Traceability)
ارائه `ValidationError` بسیار خوانا.

```
{  
  Input should be greater than 0  
  [type=greater_than,  
   input_value=-1]  
}
```

پل ارتباطی: ادغام Pydantic و Dataclass



شما مجبور نیستید بین BaseModel و is_dataclass یکی را انتخاب کنید. با استفاده از دکوراتور Pydantic روی Dataclass های استاندارد، هم رفتار ساختاری پایتون را حفظ می کنید و هم قلاب قلاب های اعتبارسنجی مانند mode='before' را به دست می آورید.

ابزارهای ساختاری سبک: TypedDict و NamedTuple



TypedDict (ساختاردهی به دیکشنری‌ها)

- برای تعریف قرارداد فایل‌های JSON خارجی.
- هیچ سربار در زمان اجرا ندارد.
- ایدئال برای بررسی ایستا روی داده‌های منعطف.



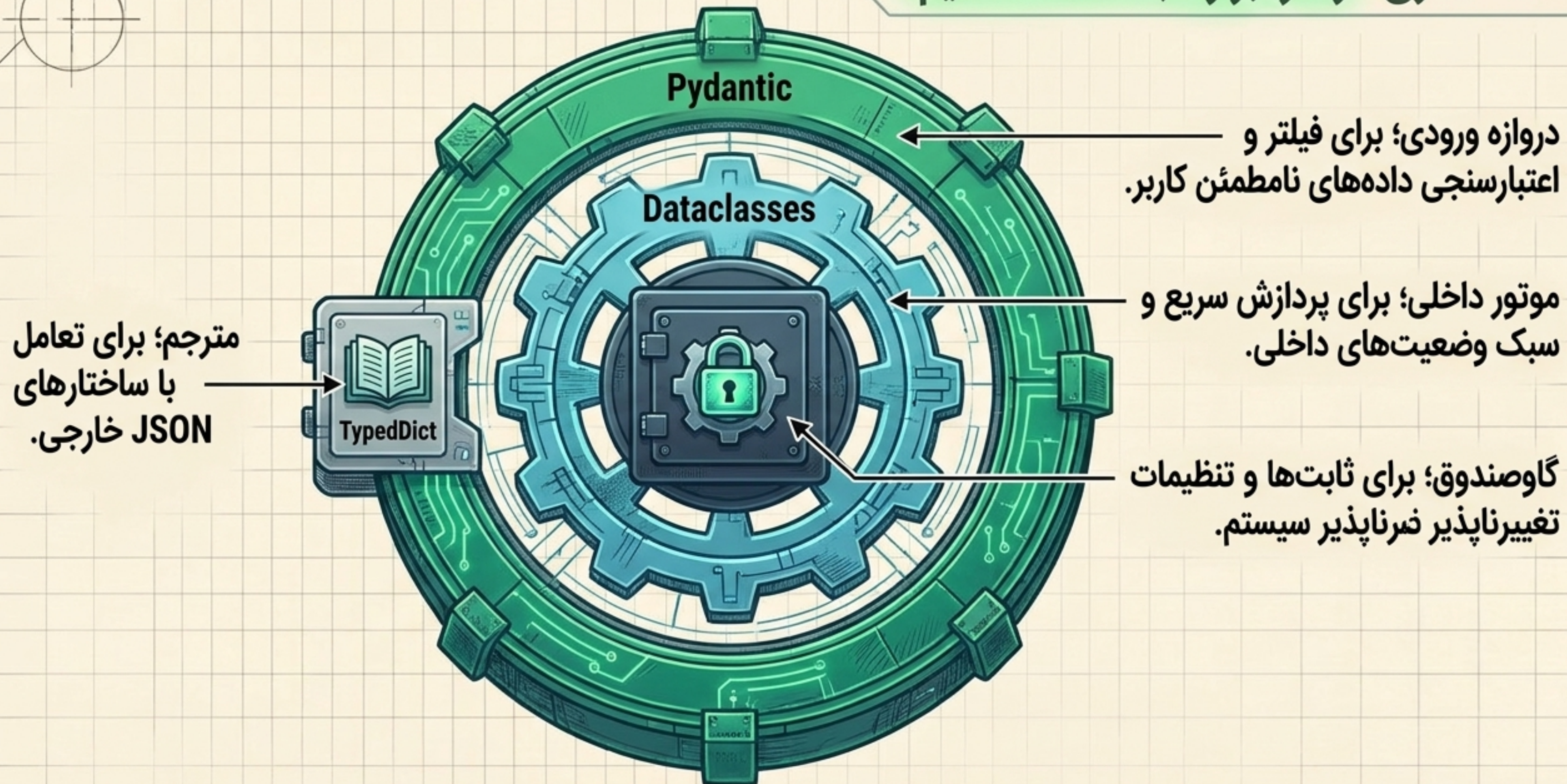
NamedTuple (پیکربندی‌های تغییرناپذیر)

- بسیار سبک، تغییرناپذیر و مبتنی بر اندیس.
- ایدئال برای تنظیمات ثابت و رویدادهای سیستمی.

ماتریس مقایسه نهایی

Pydantic	Dataclass	NamedTuple	TypedDict	ویژگی
✓	✓	✓	✓	بررسی ایستا
✓	✗	✗	✗	اعتبارسنجی در زمان اجرا
✓	✗	✗	✗	سریال سازی JSON
✓	frozen=True	✓	✗	تغییرناپذیری (Immutable)
کمی بیشتر	ناچیز	ناچیز	صفر	میزان سربار (Overhead)

نقشه معماری: از هر ابزار کجا استفاده کنیم؟



تکنیک پیشرفته: ترکیب Pydantic و TypedDict

شما نیازی ندارید راحتی ارسال دیکشنری‌ها را فدای امنیت Pydantic کنید.

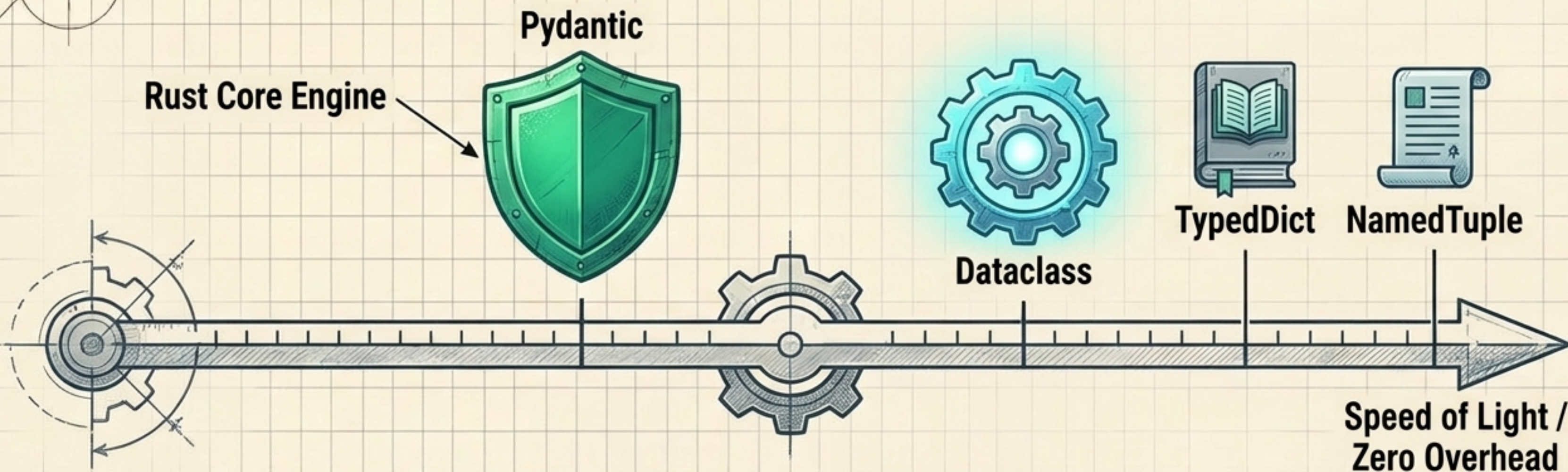
پشتیبانی از Union
برای انعطاف‌پذیری

بررسی نوع در زمان
اجرا

```
def calculate_total(order: Order | OrderTypedDict) -> float:  
    if not isinstance(order, BasoModel):  
        order = Order(**order)  
    return order.quantity * order.unit_price
```

تبدیل ایمن به مدل
سخت‌گیرانه Pydantic

ملاحظات عملکردی (Performance Overhead)



قانون طلایی: برای ورودی کاربر Pydantic غیرقابل مذاکره است.
برای ایجاد میلیون‌ها شیء در یک شبیه‌سازی داخلی، Dataclass برنده است.

اگرچه هسته Rust در نسخه ۲، Pydantic را بسیار سریع کرده است، اما همچنان برای پردازش‌های عظیم داخلی دارای سربار است.

خرد معماری: انتخاب ابزار متناسب با لایه



یک برنامه‌نویس جونیور سعی می‌کند همه مشکلات را با یک کلاس خام حل کند. یک مهندس سینیور می‌داند که Dataclass برای سرعت، Pydantic برای امنیت، TypedDict، و NamedTuple برای قراردادهای، و NamedTuple برای ثبات ساخته شده‌اند.

معماری درست از مرزبندی درست داده‌ها آغاز می‌شود.